

STUDENT RECORD SYSTEM

Software Requirements Specification & Requirements Traceability

Project	Student Record System
Team	H-11
Technology	C / C++
Problem Statement	A record-keeping system for student details such as grades and attendance.
Phase	Phase 1 – Requirements
Deliverables	Software Requirements Specification (SRS)

1. Problem Statement Analysis

Problem:

The Student Record System is intended to maintain student information such as student details, grades and attendance in an organized manner. The system should make it easier for the authorized users to add, view, search, update and delete student records.

Proposed Solution:

A menu-driven C/C++ application will be developed to maintain student records. The system will provide student management, academic record management, attendance management, searching/filtering, input validation and persistent storage.

Main Data Maintained:

- Student ID
- Student Name
- Contact Details
- Course/Department
- Semester
- Subject/Marks/Grades
- Attendance

Project Scope:

The initial implementation will focus on student CRUD operations, grades, attendance, search/filter, validation and persistent storage.

2. Feasibility Analysis

Feasibility Type	Analysis
Technical Feasibility	The system can be implemented using C/C++ with modular functions/classes and file-based persistent storage.
Economic Feasibility	The project does not require expensive hardware or software. C/C++ development tools and local file storage are sufficient.
Operational Feasibility	A simple menu-driven interface can allow users to perform student record operations easily.
Schedule Feasibility	The proposed features are suitable for implementation within the two-month mini-project duration by focusing on core student record operations.

3. Requirement List

The following requirements are derived from the given problem statement and are suitable for a two-month C/C++ mini project. The core scope is student record management, including student details, grades, attendance, searching, updating, deletion, validation, and search/filter operations.

Functional Requirements:

Req. ID	Requirement Description	Type	Design / Module	Test Case ID	Status
FR-01	The system shall allow an Administrator, Faculty member or Student to log in using valid credentials.	Functional	Authentication / User Management	TC-FR-01	Planned
FR-02	The system shall authenticate the provided credentials before granting access to the system.	Functional	Authentication / Validation	TC-FR-02	Planned
FR-03	The system shall identify the actor's role after successful authentication and provide access only to the functions permitted for that role.	Functional	Role Management / Access Control	TC-FR-03	Planned
FR-04	The system shall display a role-based main menu that provides each actor with access to the operations permitted for their role.	Functional	Menu / User Interface	TC-FR-04	Planned
FR-05	The system shall allow the Administrator to add a new student record with a unique Student ID, name, contact details, course/department and semester.	Functional	Student Management	TC-FR-05	Planned
FR-06	The system shall validate the Student ID and reject a new student record if the Student ID already exists.	Functional	Validation / Student Management	TC-FR-06	Planned
FR-07	The system shall allow the Administrator and Faculty to view student details, while a Student shall be allowed to view their own details.	Functional	Student Management	TC-FR-07	Planned
FR-08	The system shall allow the Administrator and Faculty to view stored student records according to their access permissions.	Functional	Student Management	TC-FR-08	Planned
FR-09	The system shall allow the Administrator and Faculty to search for a student using Student ID or student name.	Functional	Search & View	TC-FR-09	Planned
FR-10	The system shall allow the Administrator to update existing student information.	Functional	Student Management	TC-FR-10	Planned
FR-11	The system shall allow the Administrator to delete a student record after confirmation.	Functional	Student Management	TC-FR-11	Planned
FR-12	The system shall allow Faculty to record marks/grades for a student's subjects.	Functional	Academic Records	TC-FR-12	Planned
FR-13	The system shall allow Faculty to update the marks/grades of a student's subjects.	Functional	Academic Records	TC-FR-13	Planned
FR-14	The system shall allow Faculty to view recorded grades, and shall allow a Student to view their own grades.	Functional	Academic Records	TC-FR-14	Planned
FR-15	The system shall allow Faculty to record attendance for a student.	Functional	Attendance	TC-FR-15	Planned
FR-16	The system shall allow Faculty to update a student's attendance records.	Functional	Attendance	TC-FR-16	Planned
FR-17	The system shall allow Faculty to view attendance records, and shall allow a Student to view their own attendance.	Functional	Attendance	TC-FR-17	Planned
FR-18	The system shall allow the Administrator and Faculty to filter student records by course, department or semester.	Functional	Search & Reports	TC-FR-18	Planned
FR-19	The system shall display student records matching the specified search or filter criteria according to the actor's access permissions.	Functional	Search & Reports	TC-FR-19	Planned
FR-20	The system shall validate required fields and reject invalid data before saving the record.	Functional	Input Validation	TC-FR-20	Planned
FR-21	The system shall prompt the actor to re-enter data when invalid input is provided.	Functional	Input Validation	TC-FR-21	Planned
FR-22	The system shall save newly added or modified student records to persistent storage.	Functional	Data Storage	TC-FR-22	Planned

Req. ID	Requirement Description	Type	Design / Module	Test Case ID	Status
FR-23	The system shall retrieve stored student records when the application is started.	Functional	Data Storage	TC-FR-23	Planned
FR-24	The system shall preserve student records after the program is closed and restarted.	Functional	Data Storage	TC-FR-24	Planned
FR-25	The system shall allow the Administrator, Faculty and Student to safely exit the application through the main menu.	Functional	Menu / User Interface	TC-FR-25	Planned

Non Functional Requirements:

Req. ID	Requirement Description	Type	Design / Module	Test Case ID	Status
NFR-01	The system shall provide a simple and user-friendly menu-driven interface.	Non-Functional	User Interface	TC-NFR-01	Planned
NFR-02	The system shall complete normal add, search, update and view operations within 2 seconds for up to 1,000 student records.	Non-Functional	Performance	TC-NFR-02	Planned
NFR-03	The system shall maintain data integrity by preventing duplicate Student IDs and inconsistent student records.	Non-Functional	Data Integrity	TC-NFR-03	Planned
NFR-04	The system shall handle invalid input and runtime errors without terminating unexpectedly.	Non-Functional	Error Handling	TC-NFR-04	Planned
NFR-05	The system shall be maintainable through modular C/C++ source files and clearly defined functions/classes.	Non-Functional	Architecture / Modular Design	TC-NFR-05	Planned
NFR-06	The system shall support backup and recovery of stored student data through the selected file/database storage mechanism.	Non-Functional	Data Storage / Backup & Recovery	TC-NFR-06	Planned
NFR-07	The system shall maintain student records consistently during normal program operations and shall prevent unintended loss of saved data.	Non-Functional	Reliability / Data Storage	TC-NFR-07	Planned
NFR-08	The system shall prevent unauthorized modification of student records by restricting modification operations according to actor roles.	Non-Functional	Data Protection / Access Control	TC-NFR-08	Planned
NFR-09	The system shall be capable of running on the target computer environment using a standard C/C++ compiler.	Non-Functional	Portability	TC-NFR-09	Planned
NFR-10	The system shall provide consistent prompts, menus and messages for all student record operations.	Non-Functional	User Interface	TC-NFR-10	Planned
NFR-11	The system shall support the storage and retrieval of an increasing number of student records while maintaining acceptable performance for the intended project scope.	Non-Functional	Performance / Data Storage	TC-NFR-11	Planned
NFR-12	The system shall restrict access to functions and student information according to the authenticated actor's role.	Non-Functional	Access Control / Security	TC-NFR-12	Planned

Technical Requirements:

Req. ID	Requirement Description	Type	Design / Module	Test Case ID	Status
MTR-01	The implementation shall use C/C++ with modular functions/classes for authentication, student, academic and attendance operations.	Technical	Overall Architecture	TC-MTR-01	Planned
MTR-02	The implementation shall use persistent storage such as text, CSV, binary files or a permitted local database for student records.	Technical	Data Layer	TC-MTR-02	Planned
MTR-03	The implementation shall use unique Student IDs as the primary identifier for student records.	Technical	Student Data Model	TC-MTR-03	Planned
MTR-04	The implementation shall separate authentication, input validation, record management and storage operations into reusable modules/functions.	Technical	Software Architecture	TC-MTR-04	Planned

4.Security:

4.1 Security Objectives

ID	Security Objective
SO-01	Ensure that only authenticated users can access the Student Record System.
SO-02	Ensure that users can access student information and system operations only according to their assigned roles.

4.2 Security Requirements

ID	Security Requirement
SR-01	The system shall authenticate users using valid credentials before granting access to the system.
SR-02	The system shall restrict access to student information and operations according to the authenticated user's role.

5.Requirements Traceability Matrix:

This table maps each requirement to its architectural area, design/module, implementation file, and the planned unit, functional, system and acceptance tests. Test IDs can be used later in the Validation Specification and updated from Planned to Pass/Fail after execution.

Req. ID	Architectural Section	Design / Module	File / Implementation Unit	Unit Test ID	Functional Test ID	System Test ID	Acceptance Test ID
FR-01	Application Layer	Authentication / Login	auth.cpp / login()	UT-01	FT-01	ST-01	AT-01
FR-02	Application Layer	Credential Authentication	auth.cpp / authenticate()	UT-02	FT-02	ST-02	AT-02
FR-03	Application Layer	Role Management / Access Control	auth.cpp / role validation	UT-03	FT-03	ST-03	AT-03
FR-04	Presentation Layer	Role-Based Main Menu	main.cpp / menu functions	UT-04	FT-04	ST-04	AT-04

Req. ID	Architectural Section	Design / Module	File / Implementation Unit	Unit Test ID	Functional Test ID	System Test ID	Acceptance Test ID
FR-05	Application Layer	Student Management	student.cpp / addStudent()	UT-05	FT-05	ST-05	AT-05
FR-06	Validation Layer	Student ID Validation	validation.cpp / validateStudentID()	UT-06	FT-06	ST-06	AT-06
FR-07	Application Layer	Student Management / View	student.cpp / displayStudent()	UT-07	FT-07	ST-07	AT-07
FR-08	Application Layer	Student Record Management	student.cpp / displayAllStudents()	UT-08	FT-08	ST-08	AT-08
FR-09	Application Layer	Search & View	search.cpp / searchStudent()	UT-09	FT-09	ST-09	AT-09
FR-10	Application Layer	Student Management	student.cpp / updateStudent()	UT-10	FT-10	ST-10	AT-10
FR-11	Application Layer	Student Management	student.cpp / deleteStudent()	UT-11	FT-11	ST-11	AT-11
FR-12	Application Layer	Academic Records	grades.cpp / recordGrade()	UT-12	FT-12	ST-12	AT-12
FR-13	Application Layer	Academic Records	grades.cpp / updateGrade()	UT-13	FT-13	ST-13	AT-13
FR-14	Application Layer	Academic Records	grades.cpp / viewGrades()	UT-14	FT-14	ST-14	AT-14
FR-15	Application Layer	Attendance	attendance.cpp / recordAttendance()	UT-15	FT-15	ST-15	AT-15
FR-16	Application Layer	Attendance	attendance.cpp / updateAttendance()	UT-16	FT-16	ST-16	AT-16
FR-17	Application Layer	Attendance	attendance.cpp / viewAttendance()	UT-17	FT-17	ST-17	AT-17
FR-18	Application Layer	Search & Reports	report.cpp / filterRecords()	UT-18	FT-18	ST-18	AT-18
FR-19	Application Layer	Search & Reports	report.cpp / displayResults()	UT-19	FT-19	ST-19	AT-19
FR-20	Validation Layer	Input Validation	validation.cpp / validateInput()	UT-20	FT-20	ST-20	AT-20
FR-21	Validation Layer	Input Handling	validation.cpp / input handling	UT-21	FT-21	ST-21	AT-21
FR-22	Data Layer	Persistent Storage	storage.cpp / saveRecords()	UT-22	FT-22	ST-22	AT-22
FR-23	Data Layer	Persistent Storage	storage.cpp / loadRecords()	UT-23	FT-23	ST-23	AT-23
FR-24	Data Layer	Persistent Storage	storage.cpp / file handling	UT-24	FT-24	ST-24	AT-24
FR-25	Presentation Layer	Menu / Exit	main.cpp / exit function	UT-25	FT-25	ST-25	AT-25
NFR-01	Presentation Layer	User Interface	main.cpp / menu functions	UT-26	FT-26	ST-26	AT-26
NFR-02	Application / Data Layer	Performance	search.cpp, storage.cpp	UT-27	FT-27	ST-27	AT-27
NFR-03	Data Layer	Data Integrity	student.cpp, storage.cpp	UT-28	FT-28	ST-28	AT-28
NFR-04	Validation Layer	Error Handling	validation.cpp, main.cpp	UT-29	FT-29	ST-29	AT-29

Req. ID	Architectural Section	Design / Module	File / Implementation Unit	Unit Test ID	Functional Test ID	System Test ID	Acceptance Test ID
NFR-05	System Architecture	Modular Design	student.h/.cpp, grades.cpp, attendance.cpp	UT-30	FT-30	ST-30	AT-30
NFR-06	Data Layer	Backup & Recovery	backup.cpp, storage.cpp	UT-31	FT-31	ST-31	AT-31
NFR-07	Data Layer	Reliability	storage.cpp, student.cpp	UT-32	FT-32	ST-32	AT-32
NFR-08	Data Layer	Data Protection / Access Control	auth.cpp, student.cpp, storage.cpp	UT-33	FT-33	ST-33	AT-33
NFR-09	System Architecture	Portability	C/C++ source files	UT-34	FT-34	ST-34	AT-34
NFR-10	Presentation Layer	UI Consistency	main.cpp / menu functions	UT-35	FT-35	ST-35	AT-35
NFR-11	Data / Application Layer	Performance & Storage	storage.cpp, search.cpp	UT-36	FT-36	ST-36	AT-36
NFR-12	Application Layer	Access Control / Security	auth.cpp / role validation	UT-37	FT-37	ST-37	AT-37
MTR-01	System Architecture	C/C++ Modular Architecture	main.cpp + module .h/.cpp files	UT-38	FT-38	ST-38	AT-38
MTR-02	Data Layer	Storage Design	storage.cpp / students.dat or students.csv	UT-39	FT-39	ST-39	AT-39
MTR-03	Data Layer	Student Data Model	student.h / Student ID field	UT-40	FT-40	ST-40	AT-40
MTR-04	System Architecture	Separation of Concerns	auth.cpp, validation.cpp, student.cpp, storage.cpp	UT-41	FT-41	ST-41	AT-41
SR-01	Security/Application Layer	Authentication	Auth.cpp/authenticate()	UT-42	FT-42	ST-42	AT-42
SR-02	Security/Application Layer	Role-Based Access Control	auth.cpp / role validation	UT-43	FT-43	ST-43	AT-43

6. Validation Specification

The Validation Specification defines how the requirements of the Student Record System will be verified after implementation. Each requirement will be validated using appropriate unit, functional, system and acceptance tests. The validation activities will verify authentication, role-based access, student record management, grades, attendance, search/filter operations, input validation, persistent storage and system reliability.

Validation Levels

Test Level	Purpose
Unit Testing (UT)	Tests individual functions, classes or modules in isolation.
Functional Testing (FT)	Verifies that each implemented function satisfies its corresponding requirement.

Test Level	Purpose
System Testing (ST)	Verifies the complete integrated Student Record System.
Acceptance Testing (AT)	Verifies that the system satisfies the specified user and project requirements.

Validation Areas:

Area	Validation Focus
Authentication	Verify valid and invalid login credentials.
Role-Based Access	Verify that each actor can access only permitted functions.
Student Management	Verify adding, viewing, searching, updating and deleting student records.
Grades	Verify recording, updating and viewing grades.
Attendance	Verify recording, updating and viewing attendance.
Search and Filter	Verify valid searches, filters and no-match conditions.
Input Validation	Verify rejection of invalid, incomplete and duplicate data.
Persistent Storage	Verify that records are saved, loaded and preserved after restart.
Error Handling	Verify that invalid input does not cause unexpected termination.

7. Actors

The Student Record System has three primary actors who interact with the system according to their responsibilities.

6.1 Administrator

The Administrator is responsible for maintaining student records and managing student information in the system.

Responsibilities:

- Add new student records
- View student records
- Search student records
- Update student information
- Delete student records
- Filter student records
- Manage student record data

6.2 Faculty

The Faculty member is responsible for maintaining students' academic and attendance information.

Responsibilities:

- View student information
- Search student records
- Record student marks/grades
- Update student marks/grades
- View student grades
- Record student attendance
- Update student attendance
- View student attendance
- Filter student records when required

6.3 Student

The Student is an end user who can access and view their own academic information.

Responsibilities:

- View own student profile
- View own grades
- View own attendance
- Exit the system

Function	Administrator	Faculty	Student
Add Student	✓	—	—
View Student	✓	✓	Own record
Search Student	✓	✓	—
Update Student Information	✓	—	—
Delete Student	✓	—	—
Record Grades	—	✓	—
Update Grades	—	✓	—
View Grades	--	✓	Own grades
Record Attendance	—	✓	—
Update Attendance	—	✓	—
View Attendance	--	✓	Own attendance

Function	Administrator	Faculty	Student
Filter Student Records	✓	✓	—
Exit System	✓	✓	✓

7. Use Case Grouping Based on Actors

Administrator

Actor	Use Case
Administrator	Login
Administrator	Add Student
Administrator	View Student
Administrator	Search Student
Administrator	Update Student
Administrator	Delete Student
Administrator	Filter Student Records
Administrator	Exit System

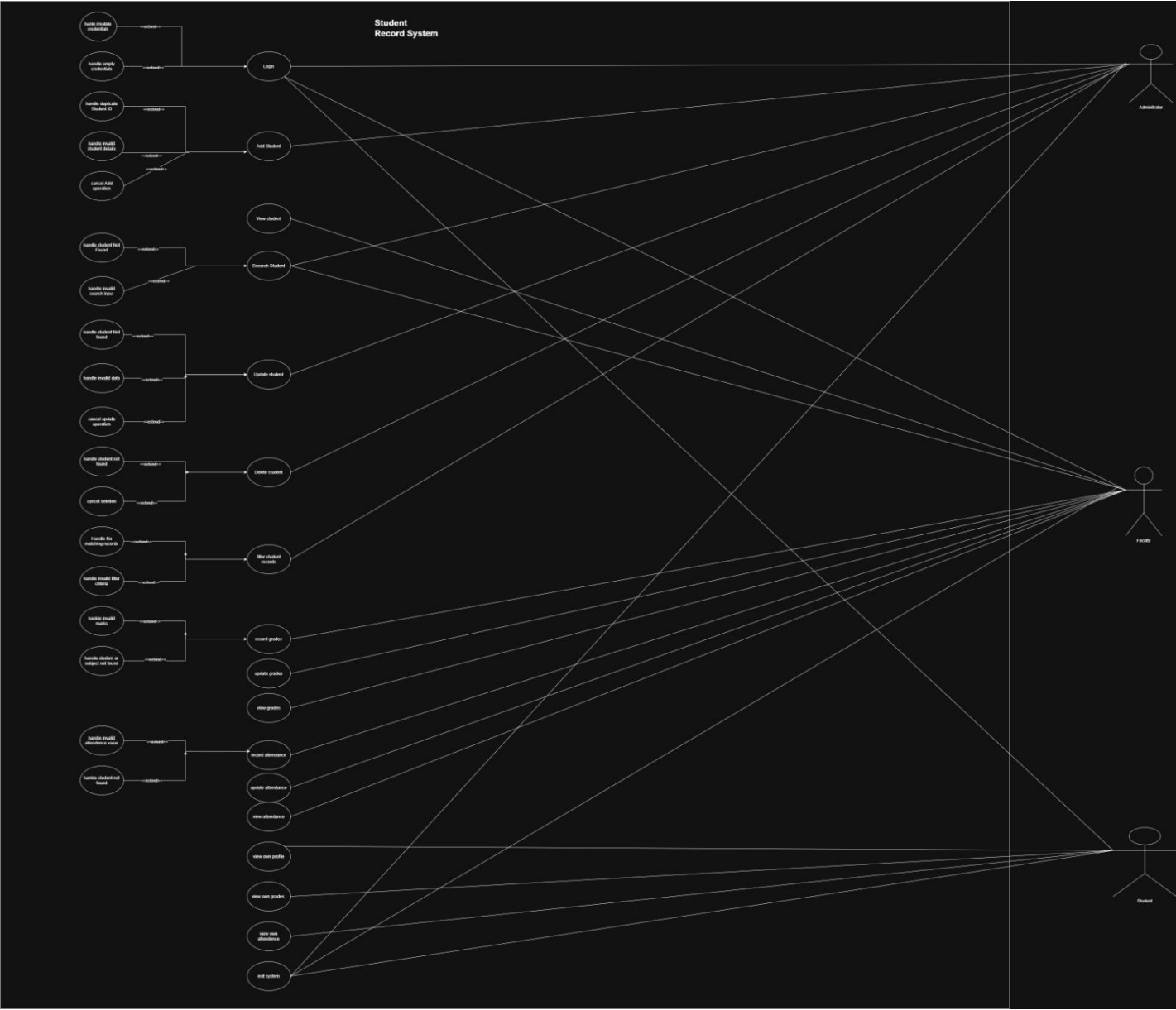
Faculty

Actor	Use Case
Faculty	Login
Faculty	View Student
Faculty	Search Student
Faculty	Record Grades
Faculty	Update Grades
Faculty	View Grades
Faculty	Record Attendance
Faculty	Update Attendance
Faculty	View Attendance
Faculty	Filter Student Records
Faculty	Exit System

Student

Actor	Use Case
Student	Login
Student	View Own Profile
Student	View Own Grades
Student	View Own Attendance
Student	Exit System

Use Case Diagram:



Alternative Flow Scenarios

UC-01: Login

AF-01 – Invalid Credentials

If the entered username or password is incorrect, the system shall display an error message and allow the actor to re-enter the credentials.

AF-02 – Empty Credentials

If the username or password field is empty, the system shall display a message requesting the actor to enter the required credentials.

UC-02: Add Student

AF-01 – Duplicate Student ID

If the entered Student ID already exists, the system shall reject the record and display a message indicating that the Student ID already exists.

AF-02 – Invalid Student Details

If one or more student details are invalid or incomplete, the system shall display an error message and request valid input.

AF-03 – Cancel Operation

If the Administrator cancels the operation before saving, the system shall discard the entered data and return to the previous menu.

UC-03: Search Student

AF-01 – Student Not Found

If no student matches the entered Student ID or student name, the system shall display a message indicating that no matching student record was found.

AF-02 – Invalid Search Input

If the search input is invalid or empty, the system shall display an appropriate message and request valid search input.

UC-04: Update Student

AF-01 – Student Not Found

If the specified Student ID does not exist, the system shall display a message indicating that the student record was not found.

AF-02 – Invalid Data

If the updated information is invalid or incomplete, the system shall reject the changes and request valid input.

AF-03 – Cancel Update

If the Administrator cancels the update operation, the system shall discard the changes and retain the existing student information.

UC-05: Delete Student

AF-01 – Student Not Found

If the specified Student ID does not exist, the system shall display a message indicating that the student record was not found.

AF-02 – User Cancels Deletion

If the Administrator cancels the deletion after the confirmation prompt, the system shall retain the student record and return to the previous menu.

UC-06: Record/Update Grades

AF-01 – Invalid Marks

If the entered marks are outside the permitted range or contain invalid input, the system shall reject the value and request valid marks.

AF-02 – Student or Subject Not Found

If the specified student or subject does not exist, the system shall display an appropriate error message and return to the grade management operation.

UC-07: Record/Update Attendance

AF-01 – Invalid Attendance Value

If the entered attendance value is outside the permitted range or contains invalid input, the system shall reject the value and request valid attendance data.

AF-02 – Student Not Found

If the specified Student ID does not exist, the system shall display a message indicating that the student record was not found.

UC-08: Filter Student Records**AF-01 – No Matching Records**

If no student records match the selected course, department or semester criteria, the system shall display a message indicating that no matching records were found.

AF-02 – Invalid Filter Criteria

If invalid filter criteria are entered, the system shall display an error message and request valid filter criteria.